

Proxy

miércoles, 6 de mayo de 2026 20:19

Es Patron estructural

Que tiene como propósito **control el acceso a un objeto mediante un intermediario** para **posponer el costo completo de su creación e inicialización** hasta que realmente necesitemos usarlos

¿Cuándo usarlo?

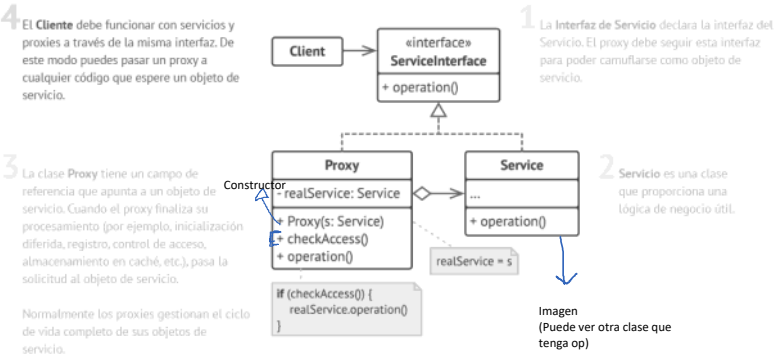
- Virtual Proxy:** demorar la construcción de un objeto hasta que realmente se necesario
Ejemplo:
En una aplicación de galería de fotos, las imágenes son muy pesadas.
En lugar de cargarlas todas de inmediato, el Proxy virtual muestra un icono o miniatura y solo carga la imagen completa cuando el usuario hace clic.
- Protection Proxy:** Restringir el acceso a un objeto por seguridad
Ejemplo:
En un sistema de cuenta bancaria, quiero proteger las cuentas para que no todo puedan acceder a la operación de depositar o extraer. Proxy verifica los permisos y delega al objeto la operación
- Remote Proxy:** representa objetos remotos en memoria local
Ejemplo:
una aplicación cliente necesita acceder a un servicio que corre en otro servidor. Proxy actúa como representante local del objeto remoto, manejando la comunicación por red (por ejemplo, llamadas RMI en Java).

Estructura

<Subject> interfaz comun que declara las operación del objeto real y proxy
<RealSubject> clase concreta que contiene logica y funcionalidad real que quiere controlar o proteger
<Proxy> clase como interfaz que actua de intermediario como virtual, protección o remote

Ventajas	Desventajas
Introduce un nivel de indirección en el acceso al objeto lo que ofrece flexibilidad	Complejidad adicional

Decorator, Proxy, Adapter → todos son wrappers estructurales.



Ejemplo de Proxy Virtual

```
public class ImageViewProxy implements
    ImageView {
    private String iPath;
    private Rectangle extent;
    private ConcreteImageView viewer;

    public ImageViewProxy(String path,
        Rectangle rec) {
        iPath = path;
        extent = rec;
    }

    public void displayImage() {
        if (viewer == null) {
            viewer = new ConcreteImageView(iPath, extent);
        }
        viewer.displayImage();
    }
}

public class ConcreteImageView
    implements ImageView {
    private Image image;
    private Rectangle extent;

    public ConcreteImageView
        (String path, Rectangle rec) {
        // Costly operation
        image = Image.load(path);
        extent = rec;
    }

    public void displayImage() {
        // Costly operation
        image.display();
    }
}
```

Ejemplo Proxy Protection

```
public class BankAccountProxy implements
    BankAccount {
    private BankAccount realAccount;
    public BankAccountProxy(BankAccount
        realAccount) {
        this.realAccount = realAccount;
    }

    public float balance() {
        if (!this.checkAccess()) {
            throw new RuntimeException("Accesso
                denegado");
        }
        return realAccount.balance();
    }

    public void deposit(float amount) {
        if (!this.checkAccess()) {
            realAccount.deposit(amount);
        }
    }

    public void withdraw(float amount) {
        if (!this.checkAccess()) {
            realAccount.withdraw(amount);
        }
    }

    private boolean checkAccess() {
        // ...
    }
}

public class RealBankAccount
    implements BankAccount {
    private float balance;

    public float balance() {
        return balance;
    }

    public void deposit(float
        amount) {
        balance += amount;
    }

    public void withdraw(float
        amount) {
        balance -= amount;
    }
}
```